NEORV32 Ascon-AEAD128 Hardware Accelerator

Design Document

A tiered, model-driven Ascon (NIST SP 800-232) accelerator, integrated as an XBUS peripheral on a fully-open RISC-V SoC and validated on silicon.

Mohammadi Masoudi Ramin

Politecnico di Milano

Companion to the Requirements & Design Document (RASD)

This document describes the *as-built* design and reflects the state validated on a Tang Nano 20K.

Contents

1 Purpose and Scope

This document describes the design of the Ascon-AEAD128 hardware accelerator and its integration into a RISC-V system-on-chip. It is a companion to the RASD: the RASD states *what* the system must do; this document states *how* it is built and *why* each structural choice was made. It reflects the design as actually realised and validated on hardware, not an idealised plan.

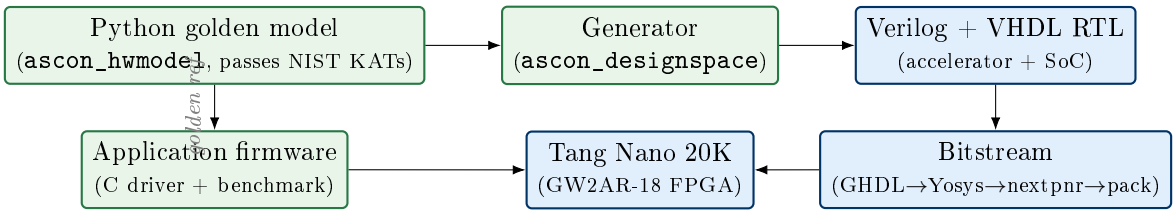
The scope covers: the generator and golden model that produce the RTL; the accelerator datapath and its five-layer module hierarchy; the register-level programming interface; the SoC integration over NEORV32's external bus (XBUS); the fully-open build toolchain; and the four-layer verification strategy. The measured on-silicon results are reported separately in the Project Report.

2 System Overview

2.1 What the system is

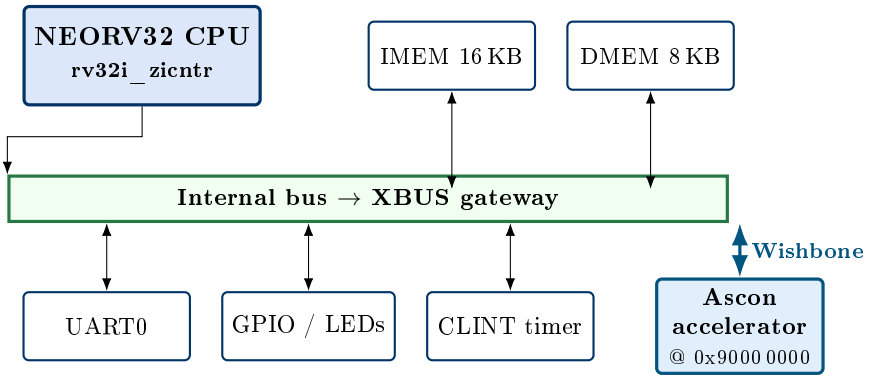
The system is a cryptographic co-processor for authenticated encryption. A RISC-V CPU (NEORV32) runs application firmware; when it needs Ascon-AEAD128 encryption or decryption, it hands the key, nonce, associated data and payload to a dedicated hardware block over a memory-mapped interface, and reads back the ciphertext and authentication tag. The accelerator performs the Ascon permutation in hardware, which is far faster than computing it in software on the same small core.

The complete signal path, from the abstract specification down to the programmed FPGA, is:



2.2 Integrated block diagram

Inside the SoC, the accelerator is one peripheral among several, reached through the bus infrastructure of the NEORV32 core.



The accelerator is deliberately *loosely coupled*: it is a bus peripheral, not a change to the CPU pipeline or instruction set. Any address the CPU issues that is not claimed by internal memory or the I/O region is routed by NEORV32's bus gateway to the external bus (XBUS), where the accelerator lives.

3 The Generator and Golden Model

3.1 Model-first methodology

The design is not written directly in RTL. The single source of truth is a typed Python model of Ascon (`ascon_hwmodel`) that implements the NIST SP 800-232 permutation and AEAD128 mode and passes the official Known-Answer Tests. From this model, a generator (`ascon_designspace`) emits the Verilog datapath and the constants (round constants, IV, S-box) as generated headers. The RTL and the golden reference therefore share one definition of "correct."

Design choice | *Golden-model-first over hand-written RTL*

A validated executable specification is generated *into* the RTL and reused as the checking oracle. This removes the most common source of crypto-hardware bugs — a subtle divergence between the implementation and the specification — because both are derived from the same model. It also makes the design parameterisable: the model describes a family of configurations, and the generator realises a chosen point.

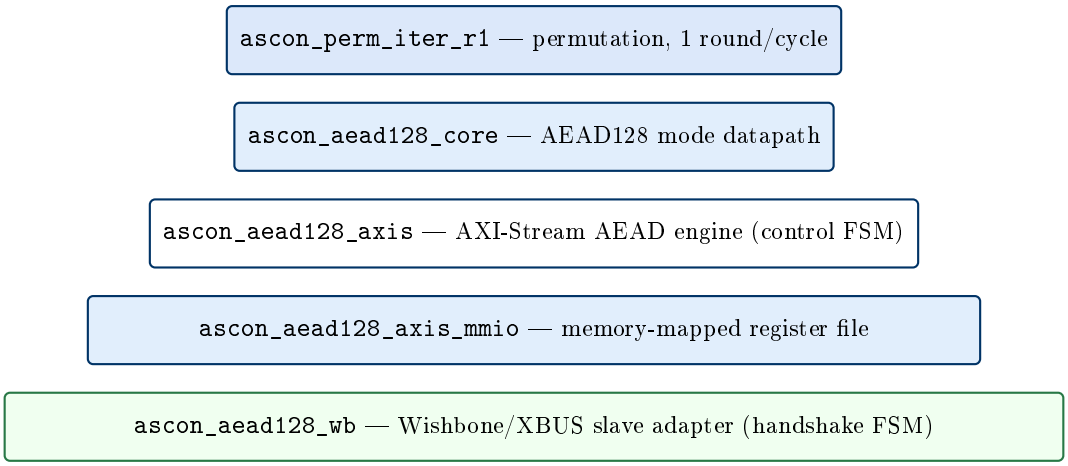
3.2 Configuration space

The model captures a design space along several architectural axes (permutation unrolling, datapath width, interface style, buffering), of which one point is realised here. The realised point is chosen for the constraints of a small FPGA: a single-round-per-cycle permutation, a 64-bit-word datapath, an AXI-Stream front end wrapped for memory-mapped access, and bounded on-chip payload buffers.

4 Accelerator Architecture

4.1 Module hierarchy

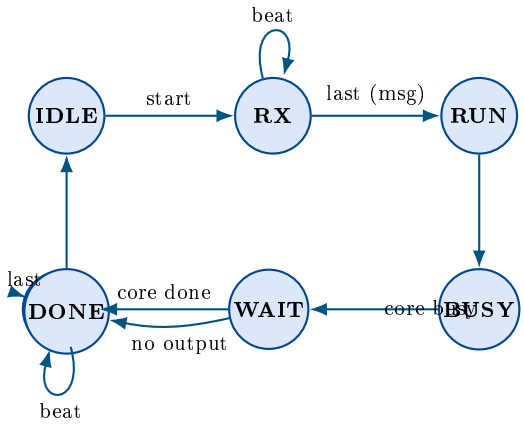
The accelerator is built as five nested layers, each with a single responsibility and each verified against the model in isolation. From the crypto core outward:



- **Permutation** — one Ascon round per clock (add round constant, substitution layer, linear diffusion). The smallest permutation variant; it is the crypto primitive shared by every AEAD phase.
- **AEAD128 core** — sequences the mode: initialise from key and nonce, absorb associated data, process the message, finalise and emit the tag, invoking the permutation the correct number of times per phase.
- **AXI-Stream engine** — a control FSM that receives the payload as a stream (associated data then message, distinguished by a sideband bit), drives the core, and emits the output stream. Its ready signal is asserted only while it is in its receive state.
- **MMIO register file** — exposes the driver-visible registers (control, status, key, nonce, lengths, data-in, data-out, tag, cycle count, error code) and bridges register writes/reads to the stream engine.
- **Wishbone adapter** — presents a Wishbone-b4 slave to NEORV32's XBUS and translates its pulsed-strobe handshake to the register file.

4.2 Engine control FSM

The stream engine's states and transitions:



Note: The engine is *start-first*: it asserts stream-ready only after leaving IDLE (i.e. after `START`). Driving payload beats before starting the engine stalls forever. This single fact dictated both the driver's write ordering and the programming sequence in Section 5.

5 Programming Interface

5.1 Register map

The accelerator occupies a small window on XBUS. Byte offsets from the base address (`0x9000 0000`):

Offset	Register	Purpose
`0x00`	CONTROL	`START` (bit 0), `DECRYPT` (bit 1), `CLEAR` (bit 8)
`0x04`	STATUS	done, error, input-ready, output-valid flags
`0x08`	MODE	AEAD variant selector
`0x0C`	CAPABILITIES	feature bits: AEAD128, cycle counter, stream data
`0x10/0x14`	AD_LEN / TEXT_LEN	payload lengths in bytes
`0x20–0x2C`	KEY0–3	128-bit key (little-endian words)
`0x30–0x3C`	NONCE0–3	128-bit nonce
`0x40`	DATA_IN	one 32-bit input word
`0x44`	DATA_IN_CT	push a beat: valid/last/keep/kind (AD vs. text)
`0x48`	DATA_OUT	one 32-bit output word
`0x4C`	DATA_OUT_(	advance the output stream
`0x60–0x6C`	TAG0–3	128-bit tag (read on encrypt; written on decrypt)
`0x78`	ERROR_COD	failure detail
`0x7C`	ABI	interface version
	CYCLE_COUNT	hardware busy-cycle counter for the last operation

5.2 Operation sequence

An encryption proceeds as follows (decryption is identical but sets `DECRYPT` and pre-loads the expected tag):

1. `CONTROL` ← `CLEAR`; write `MODE`, `KEY0..3`, `NONCE0..3`, `AD_LEN`, `TEXT_LEN`.
2. `CONTROL` ← `START` — *before* streaming, so the engine is receive-ready.
3. Stream associated data, then the message, as `DATA_IN` + `DATA_IN_CTRL` word pairs.
4. Poll `STATUS` for `done`; read `DATA_OUT` words and `TAG0..3`; read `CYCLE_COUNT` for the hardware latency.

The C driver (`ascon_accel.h`) encapsulates this entire sequence behind `ascon_accel_encrypt()` / `ascon_accel_decrypt()`, so application code never touches a register directly.

6 SoC Integration

6.1 NEORV32 configuration

The CPU is configured as a compact `rv32i` core with the cycle-counter extension (for benchmarking) and *without* the M (multiply/divide) extension. The firmware is compiled `rv32i` and provides software multiply/divide where needed, so the hardware multiplier is pure overhead; removing it frees the logic needed to fit the accelerator on the small FPGA (Section **??**).

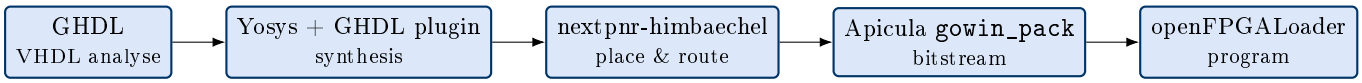
Parameter	Value / rationale
Clock	27 MHz (board oscillator, no PLL)
ISA	`rv32i` + Zicntr; M and C decisions per fit/firmware
IMEM / DMEM	16 KB / 8 KB on-chip block RAM
Boot	internal bootloader (UART upload)
Peripherals	UART0, GPIO, CLINT, XBUS
XBUS register stage	disabled (direct, minimal latency)

6.2 Bus handshake

NEORV32's XBUS drives the strobe as a single-cycle pulse and holds the cycle signal until acknowledged, sampling the acknowledge only in the cycle *after* the strobe. The adapter therefore latches the strobe, presents the access to the register file until it reports ready, and asserts the acknowledge for exactly that completing cycle. The same mechanism gives the streaming writes their natural back-pressure. This handshake was the subject of a hardware bug and is documented in detail in the Project Report.

7 Fully-Open Toolchain

Every step from source to programmed FPGA uses open-source tools, pinned by a Nix flake for reproducibility — no vendor EDA is required.



Design choice | *Fully-open, Nix-pinned build*

The flake pins the exact tool versions and, where available, uses the nixpkgs-matched Yosys GHDL plugin so the VHDL of NEORV32 and the Verilog of the accelerator synthesise together. The result is a build that is reproducible on any machine and free of proprietary tooling, at the cost of navigating the sharp edges of a bleeding-edge open flow — several of which became project challenges.

8 Verification Architecture

Correctness is established in four layers, each checking the layer below against the golden model, so that a failure is localised to one interface.

L	Layer	What it proves
1	Python golden model	Matches NIST Known-Answer Tests
2	RTL vs. model (iverilog)	Each generated module matches the model on 25 vectors
3	C driver vs. transport	The driver's register protocol is correct against an emulated peripheral
4	End-to-end co-sim	The Wishbone adapter, driven exactly as NEORV32's XBUS does, carries the full AEAD protocol correctly

The whole suite (119 tests) runs from a clean checkout and is reproduced by `nix flake check`. Layer 4 in particular replays the driver's real register ordering through the bus adapter with a NEORV32-faithful pulsed-strobe master — the regression guard for the on-chip bus behaviour.

Note: Simulation can only prove what it models. The co-simulation faithfully reproduces the XBUS handshake and the driver protocol, but the final evidence of correctness is the on-silicon run reported in the Project Report, where every case was checked on-chip against the software reference.

9 Design Choices Summary

Decision	Rationale
Model-first generation	One source of truth for RTL and reference; parameterisable
1 round / cycle permutation	Smallest crypto core; fits the GW2AR-18
Layered engine (stream → MMIO → WB)	Each layer independently verifiable; clean interfaces
XBUS peripheral	Loose coupling; no CPU pipeline changes
MMIO-word data plane	Simple, no DMA; accepts data-movement overhead
Drop M extension	Unused by `rv32i` firmware; frees area to fit
Bounded 32-byte payloads	Fits small on-chip buffers; streaming is future work
Fully-open Nix toolchain	Reproducible; no vendor lock-in

10 Known Limitations

- Payloads are bounded to 32 bytes of associated data and 32 bytes of message per call; arbitrary-length rate-block streaming is the natural extension.
- The memory-mapped, word-at-a-time data plane dominates latency for larger payloads; a DMA or wider datapath would improve throughput.
- Only the AEAD128 mode is exercised on hardware; hash/XOF modes exist in the model but are not integrated on-board.

- No side-channel or constant-time characterisation has been performed; this is a functional and performance design, not a hardened one.